Formal Verification of CategoricalCellularAutomata.jl

Simon Frost

2026-11-05

Contents

Formal verification of CategoricalCellularAutomata.jl 1

Configurations, shifts and pullbacks 3

Cellular automata over a group 4

The cartesian category of cellular automata over a group 6

Curtis–Hedlund–Lyndon 8

Linear cellular automata and the group algebra 9

Generalised cellular automata and pullbacks 10

Gluing spaces 12

Moving models between spaces 13

Maps between models 15

Quotients of models, with and without callbacks 16
Block rules . . . 21

Maps between models that read neighbours 23
Examples on a ring . . . 25
Factors . . . 26
Quotients by a section that reads neighbours . . . 27

Formal verification of CategoricalCellularAutomata.jl

This library proves, in Lean 4 with Mathlib, the laws that `CategoricalCellularAutomata.jl` relies on and that its test suite checks numerically. The definitions are the mathematical objects the Julia types implement, over any group (finite or not), with any alphabet; the Julia package works with finite groups and alphabets, so its computations are instances.

Lean	Julia	statement
`Config G A`	`Configuration`	a configuration `G → A`
`shift g x`	`shift(x, g)`	`(g · x)(h) = x(h g)`, a left action (`shift_one`, `shift_shift`)
`pullback φ x`	`pullback(φ, X)`	`x ∘ φ`, contravariantly functorial (`pullback_id`, `pullback_comp`)
`ca S μ`	`CellularAutomaton(G, S, μ)`	`τ(x)(g) = μ((x(s g))_{s ∈ S})`
`ca_shift`	`is_equivariant`	every cellular automaton commutes with the shifts

Lean	Julia	statement
`ca_eq_iff`	`σ == τ`	two automata are equal when they agree at the identity
`ca_id`, `ca_comp`	`id`, `compose`	the identity, and composites with memory set `T S`
`ca_pair`	`pair`	pairing over the disjoint union of memory sets
`CAObj G`, `IsCA`	`FullShift`, `CellularAutomaton`	alphabets, and the maps of configurations that are cellular automata
`CartesianMonoidalCategory (CAObj G)`	the Catlab `@instance` of `ThCartesianCategory`	CA(G) is a cartesian monoidal category: `compose`, `id`, `otimes`, `munit`, `proj1`, `proj2`, `pair`
`isCA_iff_shift_comm_and_local`	`is_equivariant`, `cellular_automaton`	Curtis–Hedlund–Lyndon: a cellular automaton is an equivariant map that is local at the identity
`isCA_iff_shift_comm`	`cellular_automaton`, `minimal_neighbourhood`	over a finite group, every equivariant map is a cellular automaton
`linCA a`	`LinearCA(G, coefficients; p)`	`τ(x)(g) = Σ_s a_s x(s g)`
`linCA_mul`, `linCA_pow`	`compose`, `^` for `LinearCA`	composition and powers are products in the group algebra
`gca φ S μ`	`GeneralizedCA(φ, S, μ)`	`T(x)(h) = μ((x(s φ(h)))_{s ∈ S})`
`gca_eq_pullback_ca`	`factorize`	`T = φ* ∘ τ`
`pullback_ca`	`pullback`, `SpaceMorphism`	pullback commutes with local rules: `φ* ∘ τ_{φ(S)} = τ_S ∘ φ*`
`gca_comp`	`compose`	generalised automata compose along `φ ∘ ψ`
`gluedConfigEquiv`, `exists_unique_glued_config`	`glue`	configurations of a glued space are the compatible families on its patches (a limit)
`step_restrict`, `step_eq_of_cover`	`glue`, `apply_local`	the landscape's step on a patch's interior is the patch's step
`SpaceMap`, `SpaceMap.run_pullback`	`SpaceMorphism`, `pullback(f, prob; space)`	a problem moved along a map of spaces (slots reordered, the image's rule and random numbers) runs the pullback of the original run
`CAModel`, `ModelMap`, `instCategoryCAModel`	`CAModel`, `ModelMap`, `compose`, `id`, `is_model_map`	models and the maps of cell states commuting with their rules form a category
`ModelMap.run_map`	`pushforward(f, prob)`	a map of models sends every run to the run from the mapped initial state
`SpaceMap.pullFunctor`	`pullback(f, prob; space)`, `pushforward`	moving models along a map of spaces is a functor: maps of models and of spaces commute
`Lumping`, `Lumping.model`	`quotient(model, h, section)`, `forget_layers`	the quotient's rule: lift the neighbours, apply the rule, map the result down
`Lumping.descends_iff_exists_rule`, `Lumping.rule_unique`	`is_model_map` on a quotient	`h` commutes with some rule exactly when the dynamics do not depend on what it forgets; that rule is the quotient's, whichever section is used
`Lumping.lift_comp`, `Lumping.lift_unique`	`quotient`	the universal property: maps of models that do not distinguish what `h` forgets factor through the quotient, uniquely

Lean	Julia	statement
`Lumping.liftCallback`, `Lumping.runWith_quotient`	`pushforward(f, prob; callback=:lift)`	a callback lifts when it does not depend on what is forgotten, and the quotient with it runs every run of the original, mapped down
`wholeStep`, `Lumping.runWith_wholeStep_quotient`, `Lumping.wholeStep_unique`	`pushforward(f, prob; callback=:lift_step)`	the rule and the callback as one step: when a whole step does not depend on what is forgotten, the quotient of it runs every run of the original, mapped by `h`, and is the only such step
`Lumping.wholeStep_descends`, `Lumping.liftCallback_wholeStep`, `wholeStep_strictly_more`	`callback=:lift_step` and `callback=:lift`	what descends separately descends together, with the same result, and the converse fails: a rule can write a layer that its callback reads
`BlockLumping`, `BlockLumping.blockRun_quotient`	`quotient` of a model with a `BlockRule`	block rules: the same construction on blocks of states, and the quotient runs every run of the original, mapped down
`Lumping.erased_run`	`is_model_map(f, prob)`	what a run can show: without feedback, erasing what is forgotten changes nothing kept, so a difference disproves the map
`SlidingMap`, `SlidingMap.run_map`	`SlidingBlockMap`, `pushforward`, `is_model_map`	maps that read a window around each cell send runs to runs
`instCategorySlidingModels`, `ModelMap.slidingFunctor`, `kinkMap126_not_cellwise`	`compose` of `SlidingBlockMaps` and `ModelMaps`	they form a category holding the cell-by-cell maps as a subcategory, and it is larger
`kinkMap126`, `wallsMap`, `walls_need_other_coins`	vignette 15, `Maps that read neighbours`	the boundaries of rule 126 are rule 18; the boundaries of a voter model move on the coins of two cells, whichever coins are drawn, and cannot on one
`factor_exists_iff`, `factor_unique`, `factor_isCA`	the search over elementary rules in the tests	through a surjective cellular automaton, a factor exists exactly when the rule respects the fibres, and is a unique cellular automaton
`SlidingLumping`, `SlidingLumping.run_consistent`, `SlidingLumping.stepQ_reads`, `SlidingLumping.stepQ_unique`	`quotient(model, h, section, section_window)`, `forget_layers(...; rebuild, window)`	a section that reads neighbours: the quotient's step is local, is the only one that commutes, and runs each run that stays among the consistent states
`pairLumping_run`, `pairLumping_not_descends`	`is_model_map` on such a quotient	it can hold on the consistent states and not on the others, so the local check can say no while a run says yes

Every theorem in the default target is free of `sorry`; `Audit.lean` prints the axioms each main theorem uses (only `propext`, `Classical.choice` and `Quot.sound` are expected).

Configurations, shifts and pullbacks

```
import Mathlib.Algebra.Group.Hom.Defs
```

A configuration over a group `G` with values in an alphabet `A` is a function `x : G → A` (`Configuration` in `CategoricalCellularAutomata.jl`). The group acts on configurations on the left by `(g · x)(h) = x (h * g)` (`shift(x, g)`), and a homomorphism `φ : H →* G` pulls a configuration over `G` back to one over `H` by precom-

position (`pullback(φ, X)`).

```
namespace CategoricalCellularAutomataProofs

/-- A configuration over `G` with values in `A`: the state of a cellular automaton. -/
abbrev Config (G : Type*) (A : Type*) := G → A

variable {G H K A : Type*}

section Shift

variable [Monoid G]

/-- The shift `g · x`, with `(g · x)(h) = x (h * g)` (`shift(x, g)`). -/
def shift (g : G) (x : Config G A) : Config G A := fun h ⇒ x (h * g)

@[simp] theorem shift_apply (g h : G) (x : Config G A) : shift g x h = x (h * g) := rfl

/-- The identity acts trivially. -/
@[simp] theorem shift_one (x : Config G A) : shift 1 x = x := by
  funext h; simp [shift]

/-- Shifting is a left action: `g · (g' · x) = (g g') · x`. -/
theorem shift_shift (g g' : G) (x : Config G A) : shift g (shift g' x) = shift (g * g') x := by
  funext h; simp [shift, mul_assoc]

end Shift

section Pullback

/-- The pullback `φ* x = x ∘ φ` of a configuration along a map of groups (`pullback(φ, X)`). -/
def pullback [MulOneClass H] [MulOneClass G] (φ : H →* G) (x : Config G A) : Config H A :=
  fun h ⇒ x (φ h)

@[simp] theorem pullback_apply [MulOneClass H] [MulOneClass G] (φ : H →* G) (x : Config G A)
    (h : H) : pullback φ x h = x (φ h) := rfl

/-- Pulling back along the identity changes nothing. -/
@[simp] theorem pullback_id [MulOneClass G] (x : Config G A) :
    pullback (MonoidHom.id G) x = x := rfl

/-- Pullback is contravariantly functorial: `(φ ∘ ψ)* = ψ* ∘ φ*`. -/
theorem pullback_comp [MulOneClass K] [MulOneClass H] [MulOneClass G] (φ : H →* G)
    (ψ : K →* H) (x : Config G A) :
    pullback (φ.comp ψ) x = pullback ψ (pullback φ x) := rfl

/-- Pullback intertwines the shifts: `φ* (φ(h) · x) = h · φ* x`. -/
theorem pullback_shift [Monoid H] [Monoid G] (φ : H →* G) (h : H) (x : Config G A) :
    pullback φ (shift (φ h) x) = shift h (pullback φ x) := by
  funext k; simp [pullback, shift, map_mul]

end Pullback

end CategoricalCellularAutomataProofs
```

Cellular automata over a group

```
import CategoricalCellularAutomataProofs.Configurations
```

A cellular automaton over `G` with memory set `S` and local rule `μ` maps a configuration `x` to `τ(x)(g) = μ((x(s g))_{s ∈ S})` (`CellularAutomaton(G, S, μ)`). The memory set is an ordered list of group elements, here a family `S : I → G` indexed by any type `I` (in Julia, `I = Fin (length S)`), and the local rule reads the values at `S i * g` in that order.

This file proves that every cellular automaton commutes with the shifts, that a cellular automaton is determined by its values at the identity (which is how `σ == τ` decides equality), that the identity map is a cellular automaton, that the composite of two is a cellular automaton over the product of their memory sets (`compose`), and that pairing two cellular automata is one cellular automaton over the disjoint union of their memory sets (`pair`).

```
namespace CategoricalCellularAutomataProofs

variable {G A B C : Type*} {I J : Type*}

/-- The cellular automaton with memory set `S` and local rule `μ`:
`τ(x)(g) = μ (fun i ⇒ x (S i * g))`. -/
def ca [Mul G] (S : I → G) (μ : (I → A) → B) (x : Config G A) : Config G B :=
  fun g ⇒ μ (fun i ⇒ x (S i * g))

@[simp] theorem ca_apply [Mul G] (S : I → G) (μ : (I → A) → B) (x : Config G A) (g : G) :
    ca S μ x g = μ (fun i ⇒ x (S i * g)) := rfl

section Monoid

variable [Monoid G]

/-- **Equivariance.** A cellular automaton commutes with every shift: `τ (g · x) = g · τ x`. -/
theorem ca_shift (S : I → G) (μ : (I → A) → B) (g : G) (x : Config G A) :
    ca S μ (shift g x) = shift g (ca S μ x) := by
  funext h; simp [ca, shift, mul_assoc]

/-- A shift-equivariant map is determined by its values at the identity:
`τ(x)(g) = τ(g · x)(1)`. -/
theorem apply_eq_apply_one_shift (τ : Config G A → Config G B)
    (hτ : ∀ g x, τ (shift g x) = shift g (τ x)) (x : Config G A) (g : G) :
    τ x g = τ (shift g x) 1 := by
  rw [hτ]; simp

/-- **Equality at the identity.** Two shift-equivariant maps agree when they agree at the
identity element on every configuration; this is how `σ == τ` compares cellular automata. -/
theorem eq_of_apply_one (σ τ : Config G A → Config G B)
    (hσ : ∀ g x, σ (shift g x) = shift g (σ x)) (hτ : ∀ g x, τ (shift g x) = shift g (τ x))
    (h : ∀ x, σ x 1 = τ x 1) : σ = τ := by
  funext x g
  rw [apply_eq_apply_one_shift σ hσ, apply_eq_apply_one_shift τ hτ, h]

/-- Two cellular automata are equal as maps exactly when they agree at the identity. -/
theorem ca_eq_iff {S : I → G} {T : J → G} {μ : (I → A) → B} {ν : (J → A) → B} :
    ca S μ = ca T ν ↔ ∀ x, ca S μ x 1 = ca T ν x 1 := by
  constructor
  · intro h x; rw [h]
  · intro h
    exact eq_of_apply_one _ _ (fun g x ⇒ ca_shift S μ g x) (fun g x ⇒ ca_shift T ν g x) h

/-- The identity map is the cellular automaton with memory set `{1}` that reads its one
value. -/
theorem ca_id (x : Config G A) : ca (fun _ : Unit ⇒ (1 : G)) (fun p ⇒ p ()) x = x := by
  funext g; simp [ca]

/-- **Composition.** Running `τ = ca T ν` and then `σ = ca S μ` is the cellular automaton
whose memory set is the family of products `T j * S i` (the set `T S` of `compose`) and whose
local rule applies `ν` to each block and `μ` to the results. -/
theorem ca_comp (S : I → G) (μ : (I → B) → C) (T : J → G) (ν : (J → A) → B)
    (x : Config G A) :
    ca S μ (ca T ν x) =
      ca (fun p : I × J ⇒ T p.2 * S p.1) (fun q ⇒ μ (fun i ⇒ ν (fun j ⇒ q (i, j)))) x := by
  funext g; simp [ca, mul_assoc]

/-- The composite is again equivariant, as a cellular automaton must be. -/
theorem ca_comp_shift (S : I → G) (μ : (I → B) → C) (T : J → G) (ν : (J → A) → B) (g : G)
    (x : Config G A) :
```

```
    ca S μ (ca T ν (shift g x)) = shift g (ca S μ (ca T ν x)) := by
  rw [ca_shift, ca_shift]

/-- **Pairing.** The pair of two cellular automata over the same group is one cellular
automaton whose memory set is the disjoint union of theirs (`pair`, the union `S ∪ T`). -/
theorem ca_pair (S : I → G) (μ : (I → A) → B) (T : J → G) (ν : (J → A) → C)
    (x : Config G A) (g : G) :
    ca (Sum.elim S T) (fun p ⇒ (μ (fun i ⇒ p (Sum.inl i)), ν (fun j ⇒ p (Sum.inr j)))) x g =
      (ca S μ x g, ca T ν x g) := rfl

end Monoid

end CategoricalCellularAutomataProofs
```

The cartesian category of cellular automata over a group

```
import CategoricalCellularAutomataProofs.CellularAutomata
import Mathlib.CategoryTheory.Monoidal.Cartesian.Basic
import Mathlib.Data.Finite.Prod
import Mathlib.Data.Finite.Sum
```

The package makes the cellular automata over a group `G` a Catlab instance of `ThCartesianCategory`: the objects are full shifts `A^G` (one for each alphabet `A`), the morphisms are cellular automata, composition is `compose`, the product of `A^G` and `B^G` is `(A × B)^G` (`otimes`), with `proj1`, `proj2` and `pair`, and the unit `munit()` is the one-letter alphabet.

Here the category is built in Mathlib: an object is an alphabet, a morphism is a map of configurations that *is* a cellular automaton (has a finite memory set and a local rule), and Mathlib's `CartesianMonoidalCategory.ofChosenFiniteProducts` turns the terminal object and the binary products into the cartesian monoidal structure, with all its coherence laws (the associator, unitors and braiding are then the ones the package's strict products realise).

```
namespace CategoricalCellularAutomataProofs

open CategoryTheory Limits

universe u

variable (G : Type u) [Monoid G]

/-- A map of configurations is a cellular automaton when it has a finite memory set `S` and
a local rule `μ`. -/
def IsCA {A B : Type u} (f : Config G A → Config G B) : Prop :=
  ∃ (I : Type u) (_ : Finite I) (S : I → G) (μ : (I → A) → B), f = ca S μ

variable {G}

theorem isCA_ca {A B : Type u} {I : Type u} [Finite I] (S : I → G) (μ : (I → A) → B) :
    IsCA G (ca S μ) :=
  ⟨I, inferInstance, S, μ, rfl⟩

theorem isCA_id (A : Type u) : IsCA G (id : Config G A → Config G A) :=
  ⟨PUnit, inferInstance, fun _ ⇒ 1, fun p ⇒ p PUnit.unit, by funext x g; simp [ca]⟩

theorem isCA_comp {A B C : Type u} {f : Config G A → Config G B} {g : Config G B → Config G C}
    (hf : IsCA G f) (hg : IsCA G g) : IsCA G (g ∘ f) := by
  obtain ⟨J, _, T, ν, rfl⟩ := hf
  obtain ⟨I, _, S, μ, rfl⟩ := hg
  refine ⟨I × J, inferInstance, fun p ⇒ T p.2 * S p.1,
    fun q ⇒ μ (fun i ⇒ ν (fun j ⇒ q (i, j))), ?_⟩
  funext x
  exact ca_comp S μ T ν x
```

```
/-- A cellular automaton is equivariant: it commutes with every shift. -/
theorem IsCA.shift_comm {A B : Type u} {f : Config G A → Config G B} (hf : IsCA G f) (g : G)
    (x : Config G A) : f (shift g x) = shift g (f x) := by
  obtain ⟨I, _, S, μ, rfl⟩ := hf
  exact ca_shift S μ g x

/-- An alphabet, as an object of the category of cellular automata over `G` (the full shift
`A^G`). The group is a parameter of the type, so each group has its own category. -/
@[nolint unusedArguments]
structure CAObj (G : Type u) [Monoid G] where
  /-- the alphabet -/
  α : Type u

/-- The category of cellular automata over `G`: morphisms are the maps of configurations that
are cellular automata, composed as maps. -/
instance : Category (CAObj G) where
  Hom X Y := { f : Config G X.α → Config G Y.α // IsCA G f }
  id X := ⟨id, isCA_id X.α⟩
  comp f g := ⟨g.1 ∘ f.1, isCA_comp f.2 g.2⟩

@[ext] theorem CAObj.hom_ext {X Y : CAObj G} {f g : X ⟶ Y} (h : f.1 = g.1) : f = g :=
  Subtype.ext h

@[simp] theorem CAObj.id_val (X : CAObj G) : (𝟙 X : X ⟶ X).1 = id := rfl

@[simp] theorem CAObj.comp_val {X Y Z : CAObj G} (f : X ⟶ Y) (g : Y ⟶ Z) :
    (f ≫ g).1 = g.1 ∘ f.1 := rfl

/-- The one-letter alphabet, the unit `munit()`. -/
def unitObj : CAObj G := ⟨PUnit⟩

/-- The product alphabet `A × B` (`otimes`). -/
def prodObj (X Y : CAObj G) : CAObj G := ⟨X.α × Y.α⟩

/-- The first projection `proj1`: memory set `{1}`, rule `fst`. -/
def fstHom (X Y : CAObj G) : prodObj X Y ⟶ X :=
  ⟨ca (fun _ : PUnit ⇒ 1) (fun p ⇒ (p PUnit.unit).1), isCA_ca _ _⟩

/-- The second projection `proj2`: memory set `{1}`, rule `snd`. -/
def sndHom (X Y : CAObj G) : prodObj X Y ⟶ Y :=
  ⟨ca (fun _ : PUnit ⇒ 1) (fun p ⇒ (p PUnit.unit).2), isCA_ca _ _⟩

@[simp] theorem fstHom_apply (X Y : CAObj G) (x : Config G (X.α × Y.α)) (g : G) :
    (fstHom X Y).1 x g = (x g).1 := by
  simp [fstHom, ca]

@[simp] theorem sndHom_apply (X Y : CAObj G) (x : Config G (X.α × Y.α)) (g : G) :
    (sndHom X Y).1 x g = (x g).2 := by
  simp [sndHom, ca]

/-- The pairing `pair(f, g)` of two cellular automata is a cellular automaton (over the
disjoint union of their memory sets). -/
def pairHom {W X Y : CAObj G} (f : W ⟶ X) (g : W ⟶ Y) : W ⟶ prodObj X Y :=
  ⟨fun x h ⇒ (f.1 x h, g.1 x h), by
    obtain ⟨I, _, S, μ, hS⟩ := f.2
    obtain ⟨J, _, T, ν, hT⟩ := g.2
    refine ⟨I ⊕ J, inferInstance, Sum.elim S T,
      fun p ⇒ (μ (fun i ⇒ p (Sum.inl i)), ν (fun j ⇒ p (Sum.inr j))), ?_⟩
    funext x h
    rw [hS, hT]
    exact (ca_pair S μ T ν x h).symm⟩

/-- The product cone: `(A × B)^G` with its projections is a product of `A^G` and `B^G`. -/
def prodIsLimit (X Y : CAObj G) : IsLimit (BinaryFan.mk (fstHom X Y) (sndHom X Y)) :=
  BinaryFan.IsLimit.mk _ (fun f g ⇒ pairHom f g)
    (fun f g ⇒ by ext x h; simp [pairHom])
```

```
    (fun f g ⇒ by ext x h; simp [pairHom])
    (fun f g m h₁ h₂ ⇒ by
      ext x h
      have e₁ := congrFun (congrFun (congrArg Subtype.val h₁) x) h
      have e₂ := congrFun (congrFun (congrArg Subtype.val h₂) x) h
      simp only [CAObj.comp_val, Function.comp_apply, BinaryFan.mk_fst, BinaryFan.mk_snd,
        fstHom_apply, sndHom_apply] at e₁ e₂
      show m.1 x h = (f.1 x h, g.1 x h)
      exact Prod.ext e₁ e₂)

/-- The one-letter alphabet is terminal: the only map into it is a cellular automaton with an
empty memory set. -/
def unitIsTerminal : IsTerminal (unitObj : CAObj G) :=
  IsTerminal.ofUniqueHom
    (fun X ⇒ ⟨fun _ _ ⇒ PUnit.unit, ⟨PEmpty, inferInstance, PEmpty.elim,
      fun _ ⇒ PUnit.unit, rfl⟩⟩)
    (fun X m ⇒ by ext x h; rfl)

/-- **The cartesian category CA(G).** The cellular automata over `G` form a cartesian monoidal
category, with the product of alphabets as the monoidal product and the one-letter alphabet as
its unit: the structure the package's Catlab instance of `ThCartesianCategory` asserts. -/
instance : CartesianMonoidalCategory (CAObj G) :=
  .ofChosenFiniteProducts ⟨_, unitIsTerminal⟩ (fun X Y ⇒ ⟨_, prodIsLimit X Y⟩)

end CategoricalCellularAutomataProofs
```

Curtis–Hedlund–Lyndon

```
import CategoricalCellularAutomataProofs.Category
import Mathlib.Data.Finset.Basic
import Mathlib.Data.Fintype.Basic
```

The package decides whether a map of configurations is a cellular automaton (`is_equivariant`, `cellular_automaton`, `minimal_neighbourhood`) by checking that it commutes with the shifts and that its value at the identity depends on finitely many cells. This file proves that test correct, in the uniform form that needs no topology: over any group, a map of configurations (into a nonempty alphabet) is a cellular automaton exactly when it commutes with the shifts and its value at the identity depends only on the cells of some finite set. Over a finite group the second condition always holds, so a map is a cellular automaton exactly when it commutes with the shifts, with the whole group as a memory set.

```
namespace CategoricalCellularAutomataProofs

universe u

variable {G : Type u} [Monoid G] {A B : Type u}

/-- A map whose value at the identity depends only on the cells of `S`. -/
def LocalAt (S : Finset G) (f : Config G A → Config G B) : Prop :=
  ∀ x y : Config G A, (∀ s ∈ S, x s = y s) → f x 1 = f y 1

/-- A cellular automaton's value at the identity depends only on its memory set. -/
theorem IsCA.exists_localAt [DecidableEq G] {f : Config G A → Config G B} (hf : IsCA G f) :
    ∃ S : Finset G, LocalAt S f := by
  obtain ⟨I, _, S, μ, rfl⟩ := hf
  have := Fintype.ofFinite I
  refine ⟨Finset.univ.image S, fun x y h ⇒ ?_⟩
  simp only [ca_apply, mul_one]
  congr 1
  funext i
  exact h (S i) (Finset.mem_image_of_mem S (Finset.mem_univ i))

/-- **Curtis–Hedlund–Lyndon (uniform form).** Over any group, a map of configurations into
a nonempty alphabet is a cellular automaton exactly when it commutes with the shifts and its
```

```
value at the identity depends only on a finite set of cells; that set is then a memory set. -/
theorem isCA_iff_shift_comm_and_local [DecidableEq G] [Nonempty A]
    (f : Config G A → Config G B) :
    IsCA G f ↔ (∀ g x, f (shift g x) = shift g (f x)) ∧ ∃ S : Finset G, LocalAt S f := by
  constructor
  · intro hf
    exact ⟨hf.shift_comm, hf.exists_localAt⟩
  · rintro ⟨hshift, S, hS⟩
    classical
    let ext : (S → A) → Config G A := fun p g ⇒
      if h : g ∈ S then p ⟨g, h⟩ else Classical.arbitrary A
    refine ⟨S, inferInstance, fun s ⇒ s.1, fun p ⇒ f (ext p) 1, ?_⟩
    funext x g
    rw [apply_eq_apply_one_shift f hshift x g, ca_apply]
    refine hS _ _ (fun s hs ⇒ ?_)
    simp [ext, hs, shift]

/-- **Curtis–Hedlund–Lyndon on a finite group.** A map of configurations over a finite group
is a cellular automaton exactly when it commutes with the shifts; the whole group serves as
its memory set (`cellular_automaton` recovers the rule, `minimal_neighbourhood` shrinks the
set). -/
theorem isCA_iff_shift_comm [Finite G] (f : Config G A → Config G B) :
    IsCA G f ↔ ∀ g x, f (shift g x) = shift g (f x) := by
  constructor
  · exact fun hf ⇒ hf.shift_comm
  · intro hshift
    refine ⟨G, inferInstance, id, fun p ⇒ f p 1, ?_⟩
    funext x g
    rw [apply_eq_apply_one_shift f hshift x g]
    rfl

end CategoricalCellularAutomataProofs
```

Linear cellular automata and the group algebra

```
import CategoricalCellularAutomataProofs.Category
import Mathlib.Algebra.MonoidAlgebra.Defs
import Mathlib.Tactic.Ring
```

A linear cellular automaton with coefficients `a_s` maps `x` to `τ(x)(g) = Σ_s a_s x(s g)` (`LinearCA(G, coefficients; p)`, over `GF(p)`). The package composes linear automata and takes their powers through their coefficients, as elements of the group algebra, instead of composing local rules: `compose(L, M)` (first `L`, then `M`) has the coefficients of the product `L * M`. This file proves that this is right, over any commutative semiring and any group: running `a` and then `b` is the linear automaton of `a * b`, the unit of the algebra is the identity, and a linear automaton is a cellular automaton whose memory set is the support of its coefficients.

```
namespace CategoricalCellularAutomataProofs

universe u

variable {G : Type u} [Monoid G] {R : Type u} [CommSemiring R]

/-- The linear cellular automaton with coefficients `a`: `τ(x)(g) = Σ_s a_s x(s g)`. -/
noncomputable def linCA (a : G →₀ R) (x : Config G R) : Config G R :=
  fun g ⇒ a.sum fun s r ⇒ r * x (s * g)

theorem linCA_apply (a : G →₀ R) (x : Config G R) (g : G) :
    linCA a x g = a.sum fun s r ⇒ r * x (s * g) := rfl

/-- The convolution of coefficients, `(a ⋆ b)_u = Σ_{s t = u} a_s b_t`: the product of the
group algebra (`MonoidAlgebra.mul_def`). -/
noncomputable def conv (a b : G →₀ R) : G →₀ R :=
  a.sum fun s r ⇒ b.sum fun t r' ⇒ Finsupp.single (s * t) (r * r')
```

```
/-- Running `a` and then `b` is the linear automaton of their convolution. -/
theorem linCA_conv (a b : G →₀ R) (x : Config G R) :
    linCA b (linCA a x) = linCA (conv a b) x := by
  funext g
  simp only [linCA_apply, conv]
  rw [Finsupp.sum_sum_index (fun _ ⇒ zero_mul _) (fun _ _ _ ⇒ add_mul _ _ _)]
  have h1 : ∀ s r, ((b.sum fun t r' ⇒ Finsupp.single (s * t) (r * r')).sum
      fun u r ⇒ r * x (u * g)) = b.sum fun t r' ⇒ r * r' * x (s * t * g) := by
    intro s r
    rw [Finsupp.sum_sum_index (fun _ ⇒ zero_mul _) (fun _ _ _ ⇒ add_mul _ _ _)]
    exact Finsupp.sum_congr fun t _ ⇒ Finsupp.sum_single_index (zero_mul _)
  simp only [h1, Finsupp.mul_sum]
  rw [Finsupp.sum_comm]
  refine Finsupp.sum_congr fun s _ ⇒ Finsupp.sum_congr fun t _ ⇒ ?_
  rw [mul_assoc s t g]; ring

/-- The product of the group algebra is the convolution of coefficients. -/
theorem monoidAlgebra_mul_eq_conv (a b : MonoidAlgebra R G) :
    (a * b : MonoidAlgebra R G) = (conv a b : G →₀ R) := by
  rw [MonoidAlgebra.mul_def]; rfl

/-- **Composition through the coefficients.** Running `a` and then `b` is the linear automaton
of the product `a * b` in the group algebra (`compose(L, M)`). -/
theorem linCA_mul (a b : MonoidAlgebra R G) (x : Config G R) :
    linCA b (linCA a x) = linCA (a * b : MonoidAlgebra R G) x := by
  rw [linCA_conv, monoidAlgebra_mul_eq_conv]

/-- The unit of the group algebra is the identity automaton. -/
theorem linCA_one (x : Config G R) : linCA (1 : MonoidAlgebra R G) x = x := by
  funext g
  rw [MonoidAlgebra.one_def, linCA_apply, MonoidAlgebra.single, Finsupp.sum_single_index] <;>
    simp

/-- Powers through the coefficients: running `a` `n` times is the linear automaton of `a ^ n`
(`L^n`, computed by repeated squaring of the coefficients). -/
theorem linCA_pow (a : MonoidAlgebra R G) (n : ℕ) (x : Config G R) :
    (linCA a)^[n] x = linCA (a ^ n : MonoidAlgebra R G) x := by
  induction n generalizing x with
  | zero ⇒ simp [linCA_one]
  | succ n ih ⇒
    rw [Function.iterate_succ_apply', ih, linCA_mul, ← pow_succ]

/-- A linear automaton is a cellular automaton (`CellularAutomaton(L)`): its memory set is the
support of its coefficients and its local rule the weighted sum. -/
theorem isCA_linCA (a : G →₀ R) : IsCA G (linCA a) := by
  classical
  refine ⟨a.support, inferInstance, fun s ⇒ s.1,
    fun p ⇒ ∑ s : a.support, a s.1 * p s, ?_⟩
  funext x g
  simp only [linCA_apply, ca_apply, Finsupp.sum]
  exact (Finset.sum_coe_sort a.support (fun s ⇒ a s * x (s * g))).symm

end CategoricalCellularAutomataProofs
```

Generalised cellular automata and pullbacks

```
import CategoricalCellularAutomataProofs.CellularAutomata
```

For a homomorphism `φ : H →* G`, a generalised cellular automaton maps configurations over `G` to configurations over `H` by `T(x)(h) = μ((x(s φ(h)))_{s ∈ S}) (GeneralizedCA(φ, S, μ))`.

This file proves the facts the Julia package relies on:

- `T` is the pullback along `φ` after the ordinary cellular automaton with the same memory set and rule (`factorize`);
- **pullback commutes with local rules**: pulling back the output of a cellular automaton over `G` whose memory set is the image `φ(S)` is the same as running the cellular automaton over `H` with memory set `S` on the pulled-back configuration;
- `T` intertwines the shifts along `φ`;
- two generalised automata compose to one, along the composite homomorphism, with memory set the products `S i * φ(T j)` (the set `T_φ(S)` of `compose`).

```
namespace CategoricalCellularAutomataProofs

variable {G H K A B C : Type*} {I J : Type*}

section

variable [Monoid G] [Monoid H] [Monoid K]

/-- The generalised cellular automaton along `φ` with memory set `S` and local rule `μ`. -/
def gca (φ : H →* G) (S : I → G) (μ : (I → A) → B) (x : Config G A) : Config H B :=
  fun h ⇒ μ (fun i ⇒ x (S i * φ h))

@[simp] theorem gca_apply (φ : H →* G) (S : I → G) (μ : (I → A) → B) (x : Config G A)
    (h : H) : gca φ S μ x h = μ (fun i ⇒ x (S i * φ h)) := rfl

/-- **Factorisation.** `T = φ* ∘ τ` (`factorize(T) == (φ, τ)`). -/
theorem gca_eq_pullback_ca (φ : H →* G) (S : I → G) (μ : (I → A) → B) (x : Config G A) :
    gca φ S μ x = pullback φ (ca S μ x) := rfl

/-- An ordinary cellular automaton is the generalised one over the identity homomorphism
(`GeneralizedCA(τ)`). -/
theorem gca_id (S : I → G) (μ : (I → A) → B) (x : Config G A) :
    gca (MonoidHom.id G) S μ x = ca S μ x := rfl

/-- **Pullback commutes with local rules.** For a memory set `S` in `H`, running the cellular
automaton over `G` with memory set `φ(S)` and pulling back equals pulling back and running the
cellular automaton over `H` with memory set `S`: `φ* ∘ τ_{φ(S)} = τ_S ∘ φ*`. -/
theorem pullback_ca (φ : H →* G) (S : I → H) (μ : (I → A) → B) (x : Config G A) :
    pullback φ (ca (fun i ⇒ φ (S i)) μ x) = ca S μ (pullback φ x) := by
  funext h; simp [pullback, ca, map_mul]

/-- The same, as an equality of generalised and ordinary automata:
`T_{φ, φ(S)} = τ_S ∘ φ*`. -/
theorem gca_map_eq_ca_pullback (φ : H →* G) (S : I → H) (μ : (I → A) → B)
    (x : Config G A) :
    gca φ (fun i ⇒ φ (S i)) μ x = ca S μ (pullback φ x) :=
  pullback_ca φ S μ x

/-- A generalised automaton intertwines the shifts along `φ`: `T (φ(h) · x) = h · T x`. -/
theorem gca_shift (φ : H →* G) (S : I → G) (μ : (I → A) → B) (h : H) (x : Config G A) :
    gca φ S μ (shift (φ h) x) = shift h (gca φ S μ x) := by
  funext k; simp [gca, shift, map_mul, mul_assoc]

/-- **Composition.** For `φ : H →* G` and `ψ : K →* H`, running `T = gca φ S μ` and then
`U = gca ψ T ν` is the generalised automaton along `φ ∘ ψ` whose memory set is the family of
products `S i * φ(T j)`, with the local rule applying `μ` to each block and `ν` to the
results. -/
theorem gca_comp (φ : H →* G) (S : I → G) (μ : (I → A) → B) (ψ : K →* H) (T : J → H)
    (ν : (J → B) → C) (x : Config G A) :
    gca ψ T ν (gca φ S μ x) =
      gca (φ.comp ψ) (fun p : J × I ⇒ S p.2 * φ (T p.1))
        (fun q ⇒ ν (fun j ⇒ μ (fun i ⇒ q (j, i)))) x := by
  funext k; simp [gca, map_mul, mul_assoc]

end

end CategoricalCellularAutomataProofs
```

Gluing spaces

```
import Mathlib.CategoryTheory.Limits.IsLimit
import Mathlib.CategoryTheory.Types.Basic
import Mathlib.Data.List.Basic
```

`glue` builds a landscape as the colimit of patches: each patch is a set of cells (with the slots its interior cells read), the patches share cells along their feet, and the landscape's cells are the colimit of the patches' cells. This file proves the two facts the Julia tests check numerically.

Configurations form a limit. A configuration of the landscape is a function from its cells to the alphabet, a morphism out of the colimit in `Type`. So configurations of the landscape are exactly the families of configurations of the patches that agree wherever patches share cells: the colimit of cells becomes a limit of configurations. Any colimit of cell sets will do: Catlab's colimit along an undirected wiring diagram and the package's union-find give the same one up to isomorphism.

A step restricted to a patch is the patch's step. A cell reads the cells of its slots, in slot order (the `index` attribute), and a local rule maps those values to the cell's next value. A patch's interior cells keep all their slots: the map of the patch into the landscape sends the reads of each interior cell, in order, to the reads of its image. Then one step of the landscape, read on a patch's interior, is the patch's own step on the restricted configuration (halo exchange), and the steps of patches whose interiors cover the landscape determine the landscape's step.

```
namespace CategoricalCellularAutomataProofs

open CategoryTheory Limits

universe u

section Limit

variable {J : Type u} [SmallCategory J] {F : J ⥤ Type u} {c : Cocone F}

/-- Configurations of the glued space (functions from its cells to `A`) are the compatible
families of configurations of the patches: a cocone over the patches' cells with apex `A`. -/
def gluedConfigEquiv (hc : IsColimit c) (A : Type u) :
    (c.pt → A) ≃ (F ⟶ (Functor.const J).obj A) :=
  hc.homEquiv

/-- The family a configuration of the glued space gives is its restriction to each patch. -/
theorem gluedConfigEquiv_app (hc : IsColimit c) (A : Type u) (x : c.pt → A) (j : J) :
    (gluedConfigEquiv hc A x).app j = c.ι.app j ≫ x :=
  rfl

/-- **Unique gluing.** Configurations of the patches that agree along every map of the
diagram (on shared cells) are the restrictions of exactly one configuration of the glued
space. -/
theorem exists_unique_glued_config (hc : IsColimit c) {A : Type u} (xs : ∀ j, F.obj j → A)
    (compat : ∀ {j k : J} (f : j → k), F.map f ≫ xs k = xs j) :
    ∃! x : c.pt → A, ∀ j, c.ι.app j ≫ x = xs j := by
  let s : Cocone F :=
    { pt := A
      ι := { app := xs
             naturality := fun j k f ⇒ by simpa using compat f } }
  exact hc.existsUnique s

/-- Two configurations of the glued space that agree on every patch are equal. -/
theorem glued_config_ext (hc : IsColimit c) {A : Type u} {x y : c.pt → A}
    (h : ∀ j, c.ι.app j ≫ x = c.ι.app j ≫ y) : x = y :=
  hc.hom_ext h

end Limit

section Step

variable {X P Q A : Type*}
```

```
/-- One synchronous step on a space whose cell `c` reads the cells `nbrs c` in slot order:
the local rule `μ c` maps their values to the next value of `c`. -/
def step (nbrs : X → List X) (μ : X → List A → A) (x : X → A) : X → A :=
  fun c ⇒ μ c ((nbrs c).map x)

/-- A patch of the landscape: the patch's cells `P` with a map `f` into the landscape's
cells, and its interior cells `Q`, included by `ι`, each reading `nbrsP` in slot order. The
map is a map of spaces on the interior: it sends the reads of an interior cell, in order, to
the reads of its image. -/
structure Patch (X P Q : Type*) (nbrs : X → List X) where
  /-- the patch's cells in the landscape -/
  f : P → X
  /-- the interior cells among the patch's cells -/
  ι : Q → P
  /-- the reads of each interior cell, in slot order -/
  nbrsP : Q → List P
  /-- reads are preserved, in order -/
  map_nbrs : ∀ q, (nbrsP q).map f = nbrs (f (ι q))

/-- The patch's own step on its interior, from a configuration of the patch's cells (interior
and halo); the rule of an interior cell is the rule of its image. -/
def Patch.step {nbrs : X → List X} (p : Patch X P Q nbrs) (μ : X → List A → A) (z : P → A) :
    Q → A :=
  fun q ⇒ μ (p.f (p.ι q)) ((p.nbrsP q).map z)

/-- **Halo exchange.** One step of the landscape, read on a patch's interior, is the patch's
own step on the configuration restricted to the patch. -/
theorem step_restrict {nbrs : X → List X} (p : Patch X P Q nbrs) (μ : X → List A → A)
    (x : X → A) (q : Q) :
    step nbrs μ x (p.f (p.ι q)) = p.step μ (x ∘ p.f) q := by
  simp only [step, Patch.step, ← p.map_nbrs, List.map_map]

/-- **Locality.** The landscape's next values on a patch's interior depend only on the
configuration on the patch. -/
theorem step_local {nbrs : X → List X} (p : Patch X P Q nbrs) (μ : X → List A → A)
    {x y : X → A} (h : x ∘ p.f = y ∘ p.f) (q : Q) :
    step nbrs μ x (p.f (p.ι q)) = step nbrs μ y (p.f (p.ι q)) := by
  rw [step_restrict, step_restrict, h]

/-- **The patches determine the step.** If the interiors of a family of patches cover the
landscape, the landscape's step is the unique configuration whose value on each patch's
interior is that patch's step on the restriction. -/
theorem step_eq_of_cover {ι' : Type*} {P Q : ι' → Type*} {nbrs : X → List X}
    (ps : ∀ k, Patch X (P k) (Q k) nbrs) (μ : X → List A → A)
    (cover : ∀ c, ∃ k q, (ps k).f ((ps k).ι q) = c) (x y : X → A)
    (hy : ∀ k q, y ((ps k).f ((ps k).ι q)) = (ps k).step μ (x ∘ (ps k).f) q) :
    y = step nbrs μ x := by
  funext c
  obtain ⟨k, q, rfl⟩ := cover c
  rw [hy, step_restrict]

end Step

end CategoricalCellularAutomataProofs
```

Moving models between spaces

```
import CategoricalCellularAutomataProofs.Gluing
```

A space is a set of cells `X`, each reading its neighbours `nbrs c` in slot order (a list, so graphs, whose cells have different numbers of neighbours, are included). A map of spaces `φ : Y → X` preserves neighbours up to a reordering of each cell's slots: `ρ y` puts the images of `y`'s neighbours in the slot order of `φ y`. A reordering moves list entries by

position, so it commutes with relabelling them (`ρ_natural`): a natural transformation of the list functor, such as the identity (a covering) or reversal (a one-dimensional fold).

The package's `pullback(f, prob; space)` moves a problem along such a map: each cell `y` of the new space runs the rule of its image `φ y`, reading its neighbours reordered by `ρ y`, and draws the random numbers of `φ y`. A rule that depends on the cell and the time (as a rule drawing counter-based random numbers keyed by cell and step does) is a family `μ t c : List A → A`. The pulled-back run is the pullback of the run, step by step, for every such rule: no symmetry of the rule is needed.

```
namespace CategoricalCellularAutomataProofs

universe u

variable {X Y A : Type u}

/-- The run of a time-dependent rule: `run nbrs μ x n` is the configuration after `n`
synchronous steps from `x`, step `t` applying the rule `μ t`. -/
def run (nbrs : X → List X) (μ : ℕ → X → List A → A) (x : X → A) : ℕ → X → A
  | 0 ⇒ x
  | n + 1 ⇒ step nbrs (μ n) (run nbrs μ x n)

/-- A map of spaces `φ : Y → X` with slot reorderings `ρ`. -/
structure SpaceMap (Y X : Type u) (nbrsY : Y → List Y) (nbrsX : X → List X) where
  /-- where each cell goes -/
  φ : Y → X
  /-- the reordering of each cell's slots, at every type of values -/
  ρ : Y → ∀ {B : Type u}, List B → List B
  /-- a reordering moves entries by position, so it commutes with relabelling them -/
  ρ_natural : ∀ (y : Y) {B C : Type u} (g : B → C) (l : List B), ρ y (l.map g) = (ρ y l).map g
  /-- neighbours are preserved: the images of `y`'s neighbours, reordered, are `φ y`'s -/
  map_nbrs : ∀ y, ρ y ((nbrsY y).map φ) = nbrsX (φ y)

variable {nbrsY : Y → List Y} {nbrsX : X → List X}

/-- The rule pulled back along a map of spaces: the cell `y` runs the rule of `φ y` on its
neighbours' values in `φ y`'s slot order. -/
def SpaceMap.pullRule (f : SpaceMap Y X nbrsY nbrsX) (μ : X → List A → A) : Y → List A → A :=
  fun y l ⇒ μ (f.φ y) (f.ρ y l)

/-- One step of the pulled-back rule from a pulled-back configuration is the pullback of one
step. -/
theorem SpaceMap.step_pullback (f : SpaceMap Y X nbrsY nbrsX) (μ : X → List A → A)
    (x : X → A) : step nbrsY (f.pullRule μ) (x ∘ f.φ) = step nbrsX μ x ∘ f.φ := by
  funext y
  simp only [step, SpaceMap.pullRule, Function.comp_apply]
  rw [← List.map_map, f.ρ_natural, f.map_nbrs]

/-- **Moving a model along a map of spaces.** The run of the pulled-back time-dependent rule
from the pulled-back initial state is, at every step, the pullback of the run. -/
theorem SpaceMap.run_pullback (f : SpaceMap Y X nbrsY nbrsX) (μ : ℕ → X → List A → A)
    (x : X → A) (n : ℕ) :
    run nbrsY (fun t ⇒ f.pullRule (μ t)) (x ∘ f.φ) n = run nbrsX μ x n ∘ f.φ := by
  induction n with
  | zero ⇒ rfl
  | succ n ih ⇒ simp only [run, ih, SpaceMap.step_pullback]

/-- A covering: a map that preserves neighbours slot by slot (no reordering). -/
def SpaceMap.ofSlotwise (φ : Y → X) (h : ∀ y, (nbrsY y).map φ = nbrsX (φ y)) :
    SpaceMap Y X nbrsY nbrsX where
  φ := φ
  ρ _ _ l := l
  ρ_natural _ _ _ _ _ := rfl
  map_nbrs := h

/-- A fold that reverses the slots of some cells (on a line with neighbours `[left, right]`,
the mirrored half reads them the other way round), as `reflection_fold` builds. -/
```

```
def SpaceMap.ofReversing (φ : Y → X) (flip : Y → Bool)
    (h : ∀ y, (if flip y then List.reverse else id) ((nbrsY y).map φ) = nbrsX (φ y)) :
    SpaceMap Y X nbrsY nbrsX where
  φ := φ
  ρ y _ l := if flip y then l.reverse else l
  ρ_natural y _ _ g l := by cases flip y <;> simp [List.map_reverse]
  map_nbrs y := by simpa [apply_ite (fun F : List X → List X ⇒ F ((nbrsY y).map φ))] using h y

end CategoricalCellularAutomataProofs
```

Maps between models

```
import CategoricalCellularAutomataProofs.Spaces
import Mathlib.CategoryTheory.Functor.Basic
```

A model on a space (cells `X`, each reading its neighbours in slot order) is a set of cell states `A` with a rule `μ t c : List A → A` for each time `t` and cell `c` (a rule drawing counter-based random numbers keyed by cell and step is such a family). A map of models `M → N` is a map of cell states `h : M.A → N.A` that commutes with the rules: the next state of `M`, mapped by `h`, is the next state of `N` from the mapped neighbours, at every time and cell. It forgets what does not feed back (a lineage label carried along, the colours of the immigration game), and it sends every run of `M` to the run of `N` from the mapped initial state (`ModelMap.run_map`).

The package's `ModelMap(dom, codom, h)` is such a map (its parameter map is absorbed here into the rules, which are the models' rules at given parameters), `is_model_map` checks the commuting condition on every local configuration, and `pushforward(f, prob)` maps a problem. Models and their maps form a category, and pulling back along a map of spaces is a functor between the categories of models on the two spaces (`SpaceMap.pullFunctor`): maps of models and maps of spaces commute.

```
namespace CategoricalCellularAutomataProofs

open CategoryTheory

universe u

variable {X Y : Type u}

/-- A model on the cells `X`: cell states and a time- and cell-dependent rule. -/
structure CAModel (X : Type u) where
  /-- the cell states -/
  A : Type u
  /-- the rule at time `t` and cell `c`, reading the neighbours' states in slot order -/
  μ : ℕ → X → List A → A

/-- A map of models: a map of cell states commuting with the rules. -/
@[ext]
structure ModelMap (M N : CAModel X) where
  /-- the map of cell states, applied cell by cell -/
  h : M.A → N.A
  /-- the next state, mapped, is the next state from the mapped neighbours -/
  comm : ∀ t c l, h (M.μ t c l) = N.μ t c (l.map h)

variable {nbrs : X → List X}

/-- One step of the codomain from a mapped configuration is the mapped step. -/
theorem ModelMap.step_map {M N : CAModel X} (f : ModelMap M N) (t : ℕ) (x : X → M.A) :
    step nbrs (N.μ t) (f.h ∘ x) = f.h ∘ step nbrs (M.μ t) x := by
  funext c
  simp only [step, Function.comp_apply, f.comm, List.map_map]

/-- **Maps of models send runs to runs.** The run of the codomain from the mapped initial
state is, at every step, the run of the domain mapped cell by cell. -/
theorem ModelMap.run_map {M N : CAModel X} (f : ModelMap M N) (x : X → M.A) (n : ℕ) :
    run nbrs N.μ (f.h ∘ x) n = f.h ∘ run nbrs M.μ x n := by
```

```
  induction n with
  | zero ⇒ rfl
  | succ n ih ⇒ simp only [run, ih, ModelMap.step_map]

/-- The identity map of a model. -/
def ModelMap.id (M : CAModel X) : ModelMap M M where
  h a := a
  comm t c l := by simp

/-- A map of models followed by another. -/
def ModelMap.comp {M N P : CAModel X} (f : ModelMap M N) (g : ModelMap N P) :
    ModelMap M P where
  h := g.h ∘ f.h
  comm t c l := by simp only [Function.comp_apply, f.comm, g.comm, List.map_map]

/-- Models on `X` and the maps between them form a category. -/
instance instCategoryCAModel : Category.{u} (CAModel X) where
  Hom M N := ModelMap M N
  id M := ModelMap.id M
  comp f g := ModelMap.comp f g
  id_comp _ := rfl
  comp_id _ := rfl
  assoc _ _ _ := rfl

variable {nbrsY : Y → List Y} {nbrsX : X → List X}

/-- A model moved along a map of spaces: each cell runs its image's rule on its neighbours in
the image's slot order (`pullback(f, prob; space)`). -/
def SpaceMap.pullModel (f : SpaceMap Y X nbrsY nbrsX) (M : CAModel X) : CAModel Y where
  A := M.A
  μ t := f.pullRule (M.μ t)

/-- A map of models is a map of the models moved along a map of spaces: the slot reordering
moves entries by position, so it commutes with mapping them. -/
def SpaceMap.pullMap (f : SpaceMap Y X nbrsY nbrsX) {M N : CAModel X} (F : ModelMap M N) :
    ModelMap (f.pullModel M) (f.pullModel N) where
  h := F.h
  comm t y l := by
    simp only [SpaceMap.pullModel, SpaceMap.pullRule, F.comm, f.ρ_natural]

/-- **Moving models along a map of spaces is a functor** from the models on `X` to the models
on `Y`: maps of models and maps of spaces commute. -/
def SpaceMap.pullFunctor (f : SpaceMap Y X nbrsY nbrsX) : CAModel X ⥤ CAModel Y where
  obj := f.pullModel
  map F := f.pullMap F
  map_id _ := rfl
  map_comp _ _ := rfl

end CategoricalCellularAutomataProofs
```

Quotients of models, with and without callbacks

```
import CategoricalCellularAutomataProofs.Models
```

A lumping of a model `M` is a map of cell states `h : M.A → B` with a section `s` (`h (s b) = b`): a way to forget something, and a way to fill it back in. The package's `quotient(model, h, section)` builds the quotient model from `M`'s own rule: lift the neighbours with `s`, apply the rule, and map the result down with `h`.

That rule exists for every lumping. It makes `h` a map of models exactly when the next state, mapped by `h`, does not depend on what `h` forgets (`Lumping.Descends`), and then it is the only rule that does, whichever section was used (`Lumping.rule_unique`), and the quotient has the universal property of a quotient: every map of models that does not distinguish what `h` forgets factors through it, uniquely (`Lumping.lift_comp`, `Lumping.lift_unique`).

A callback acts on the whole configuration, not cell by cell, and runs after each step. It lifts in the same way, `y`

↦ h ∘ κ (s ∘ y), under the same kind of condition (`Lumping.CallbackDescends`). With both conditions the quotient, with its lifted callback, runs every run of the original mapped by h (`Lumping.runWith_quotient`). The package's `pushforward(f, prob; callback=:lift)` is this construction.

Mapping down between the rule and the callback loses whatever lives only inside a step: a layer the rule writes, the callback reads, and the next rule overwrites. The rule and the callback can instead be treated as one step, `wholeStep`. The same construction applies to it: lift the configuration, take the step, map down. `Lumping.wholeStep_descends` and `Lumping.liftCallback_wholeStep` show this extends the separate construction, and `wholeStep_strictly_more` shows the extension is strict. The package's `pushforward(f, prob; callback=:lift_step)` is this construction.

Block rules, whose cells are written a block at a time, get the same construction one level up (`BlockLumping`). Together with `Lumping` this covers every rule `quotient` accepts.

The last theorem of `Lumping` says what erasing a layer after every step can show. If h forgets without feedback, the erased run agrees with the original on everything kept (`Lumping.erased_run`). So a difference between the two disproves the map; agreement on one problem does not prove it.

```
namespace CategoricalCellularAutomataProofs

open CategoryTheory

universe u

variable {X : Type u} {nbrs : X → List X}

/-- The run of a time-dependent rule with a callback: each step applies the rule, then the
callback `κ n` to the whole configuration. -/
def runWith {A : Type u} (nbrs : X → List X) (μ : ℕ → X → List A → A)
    (κ : ℕ → (X → A) → X → A) (x : X → A) : ℕ → X → A
  | 0 ⇒ x
  | n + 1 ⇒ κ n (step nbrs (μ n) (runWith nbrs μ κ x n))

/-- A map of models that commutes with two callbacks sends runs with the first to runs with the
second. -/
theorem ModelMap.runWith_map {M N : CAModel X} (F : ModelMap M N)
    (κM : ℕ → (X → M.A) → X → M.A) (κN : ℕ → (X → N.A) → X → N.A)
    (hκ : ∀ n x, F.h ∘ κM n x = κN n (F.h ∘ x)) (x : X → M.A) (n : ℕ) :
    runWith nbrs N.μ κN (F.h ∘ x) n = F.h ∘ runWith nbrs M.μ κM x n := by
  induction n with
  | zero ⇒ rfl
  | succ n ih ⇒ simp only [runWith, ih, ModelMap.step_map, hκ]

/-- One step with its callback, as a single map of configurations. -/
def wholeStep {A : Type u} (nbrs : X → List X) (μ : ℕ → X → List A → A)
    (κ : ℕ → (X → A) → X → A) (n : ℕ) (x : X → A) : X → A :=
  κ n (step nbrs (μ n) x)

/-- The run of any time-dependent map of configurations. -/
def iterateStep {A : Type u} (G : ℕ → (X → A) → X → A) (x : X → A) : ℕ → X → A
  | 0 ⇒ x
  | n + 1 ⇒ G n (iterateStep G x n)

/-- A run with a callback is the run of its whole steps. -/
theorem runWith_eq_iterateStep {A : Type u} (μ : ℕ → X → List A → A)
    (κ : ℕ → (X → A) → X → A) (x : X → A) (n : ℕ) :
    runWith nbrs μ κ x n = iterateStep (wholeStep nbrs μ κ) x n := by
  induction n with
  | zero ⇒ rfl
  | succ n ih ⇒ simp only [runWith, iterateStep, wholeStep, ih]

/-- A way to forget something about cell states of `M`, and to fill it back in. -/
structure Lumping (M : CAModel X) (B : Type u) where
  /-- the map of cell states -/
  h : M.A → B
  /-- the lift of a cell state -/
  s : B → M.A
```

```
  /-- lifting and mapping down changes nothing -/
  h_s : ∀ b, h (s b) = b

namespace Lumping

variable {M N : CAModel X} {B : Type u} (q : Lumping M B)

theorem h_comp_s : q.h ∘ q.s = _root_.id := funext q.h_s

theorem map_h_map_s (l : List B) : (l.map q.s).map q.h = l := by
  rw [List.map_map, q.h_comp_s, List.map_id]

/-- The quotient model: lift the neighbours, apply `M`'s rule, map the result down. -/
def model : CAModel X where
  A := B
  μ t c l := q.h (M.μ t c (l.map q.s))

/-- The next state, mapped by `h`, does not depend on what `h` forgets. -/
def Descends : Prop :=
  ∀ t c (l l' : List M.A), l.map q.h = l'.map q.h → q.h (M.μ t c l) = q.h (M.μ t c l')

/-- When the dynamics do not depend on what is forgotten, `h` is a map onto the quotient. -/
def map (hd : q.Descends) : ModelMap M q.model where
  h := q.h
  comm t c l := hd t c l ((l.map q.h).map q.s) (q.map_h_map_s _).symm

/-- **Any rule that `h` commutes with is the quotient's**, whichever section was used. -/
theorem rule_unique {μ' : ℕ → X → List B → B}
    (comm : ∀ t c l, q.h (M.μ t c l) = μ' t c (l.map q.h)) : μ' = q.model.μ := by
  funext t c l
  show μ' t c l = q.h (M.μ t c (l.map q.s))
  rw [comm, q.map_h_map_s]

/-- **`h` commutes with some rule exactly when the dynamics do not depend on what it forgets.**
This is what `is_model_map` tests for a quotient. -/
theorem descends_iff_exists_rule :
    q.Descends ↔ ∃ μ' : ℕ → X → List B → B, ∀ t c l, q.h (M.μ t c l) = μ' t c (l.map q.h) := by
  constructor
  · intro hd
    exact ⟨q.model.μ, (q.map hd).comm⟩
  · rintro ⟨μ', comm⟩ t c l l' e
    rw [comm, comm, e]

/-- Two sections of the same map give the same quotient rule. -/
theorem rule_eq_of_section (q' : Lumping M B) (hh : q'.h = q.h) (hd : q.Descends) :
    q'.model.μ = q.model.μ := by
  apply q.rule_unique
  intro t c l
  have e : l.map q.h = ((l.map q.h).map q'.s).map q.h := by
    have := q'.map_h_map_s (l.map q.h)
    rw [hh] at this
    exact this.symm
  show q.h (M.μ t c l) = q'.h (M.μ t c ((l.map q.h).map q'.s))
  rw [hh]
  exact hd t c _ _ e

/-- The map out of the quotient induced by a map of models that does not distinguish what `h`
forgets. -/
def lift (F : ModelMap M N) (hF : ∀ a a', q.h a = q.h a' → F.h a = F.h a') :
    ModelMap q.model N where
  h b := F.h (q.s b)
  comm t c l := by
    show F.h (q.s (q.h (M.μ t c (l.map q.s)))) = N.μ t c (l.map fun b ⇒ F.h (q.s b))
    rw [hF _ _ (q.h_s _), F.comm, List.map_map]
    rfl
```

```
/-- **Universal property, existence.** The map factors through the quotient. -/
theorem lift_comp (hd : q.Descends) (F : ModelMap M N)
    (hF : ∀ a a', q.h a = q.h a' → F.h a = F.h a') :
    (q.map hd).comp (q.lift F hF) = F := by
  apply ModelMap.ext
  funext a
  exact hF _ _ (q.h_s _)

/-- **Universal property, uniqueness.** The factorisation is the only one. -/
theorem lift_unique (hd : q.Descends) (F : ModelMap M N)
    (hF : ∀ a a', q.h a = q.h a' → F.h a = F.h a') (G : ModelMap q.model N)
    (hG : (q.map hd).comp G = F) : G = q.lift F hF := by
  apply ModelMap.ext
  funext b
  have e : F.h (q.s b) = G.h (q.h (q.s b)) := by rw [← hG]; rfl
  show G.h b = F.h (q.s b)
  rw [e, q.h_s]

/-- The callback of the quotient: lift the configuration, apply the callback, map down. -/
def liftCallback (κ : ℕ → (X → M.A) → X → M.A) : ℕ → (X → B) → X → B :=
  fun n y ⇒ q.h ∘ κ n (q.s ∘ y)

/-- What a callback leaves, mapped by `h`, does not depend on what `h` forgets. -/
def CallbackDescends (κ : ℕ → (X → M.A) → X → M.A) : Prop :=
  ∀ n (x x' : X → M.A), q.h ∘ x = q.h ∘ x' → q.h ∘ κ n x = q.h ∘ κ n x'

theorem h_comp_s_comp (y : X → B) : q.h ∘ (q.s ∘ y) = y := by
  funext c
  exact q.h_s (y c)

/-- When the callback does not depend on what is forgotten, `h` commutes with the lifted
callback. -/
theorem liftCallback_comm {κ : ℕ → (X → M.A) → X → M.A} (hκ : q.CallbackDescends κ) (n : ℕ)
    (x : X → M.A) : q.h ∘ κ n x = q.liftCallback κ n (q.h ∘ x) :=
  hκ n x (q.s ∘ (q.h ∘ x)) (q.h_comp_s_comp (q.h ∘ x)).symm

/-- **`h` commutes with some callback exactly when the callback does not depend on what it
forgets.** -/
theorem callbackDescends_iff_exists {κ : ℕ → (X → M.A) → X → M.A} :
    q.CallbackDescends κ ↔
      ∃ κ' : ℕ → (X → B) → X → B, ∀ n x, q.h ∘ κ n x = κ' n (q.h ∘ x) := by
  constructor
  · intro hκ
    exact ⟨q.liftCallback κ, q.liftCallback_comm hκ⟩
  · rintro ⟨κ', comm⟩ n x x' e
    rw [comm, comm, e]

/-- **Any callback that `h` commutes with is the lifted one.** -/
theorem callback_unique {κ : ℕ → (X → M.A) → X → M.A} {κ' : ℕ → (X → B) → X → B}
    (comm : ∀ n x, q.h ∘ κ n x = κ' n (q.h ∘ x)) : κ' = q.liftCallback κ := by
  funext n y
  show κ' n y = q.h ∘ κ n (q.s ∘ y)
  rw [comm, q.h_comp_s_comp]

/-- **The quotient with its lifted callback runs every run of the original, mapped by `h`.** -/
theorem runWith_quotient (hd : q.Descends) {κ : ℕ → (X → M.A) → X → M.A}
    (hκ : q.CallbackDescends κ) (x : X → M.A) (n : ℕ) :
    runWith nbrs q.model.μ (q.liftCallback κ) (q.h ∘ x) n = q.h ∘ runWith nbrs M.μ κ x n :=
  (q.map hd).runWith_map κ (q.liftCallback κ) (q.liftCallback_comm hκ) x n

/-- The quotient of any map of configurations that does not depend on what `h` forgets runs its
runs mapped by `h`. `liftCallback` and `CallbackDescends` do not use that `G` is a callback, so
what they say about callbacks, existence and uniqueness included, holds for whole steps. -/
theorem iterateStep_quotient {G : ℕ → (X → M.A) → X → M.A} (hG : q.CallbackDescends G)
    (x : X → M.A) (n : ℕ) :
    iterateStep (q.liftCallback G) (q.h ∘ x) n = q.h ∘ iterateStep G x n := by
```

```
  induction n with
  | zero ⇒ rfl
  | succ n ih ⇒
    simp only [iterateStep]
    rw [ih]
    exact (q.liftCallback_comm hG n _).symm

/-- **Treating the rule and the callback as one step, the quotient runs every run of the original,
mapped by `h`**, when what a whole step leaves does not depend on what `h` forgets. Neither the
rule nor the callback need satisfy that alone. -/
theorem runWith_wholeStep_quotient {κ : ℕ → (X → M.A) → X → M.A}
    (hG : q.CallbackDescends (wholeStep nbrs M.μ κ)) (x : X → M.A) (n : ℕ) :
    iterateStep (q.liftCallback (wholeStep nbrs M.μ κ)) (q.h ∘ x) n =
      q.h ∘ runWith nbrs M.μ κ x n := by
  rw [runWith_eq_iterateStep]
  exact q.iterateStep_quotient hG x n

/-- **Any whole step that `h` commutes with is the lifted one.** -/
theorem wholeStep_unique {κ : ℕ → (X → M.A) → X → M.A} {G' : ℕ → (X → B) → X → B}
    (comm : ∀ n x, q.h ∘ wholeStep nbrs M.μ κ n x = G' n (q.h ∘ x)) :
    G' = q.liftCallback (wholeStep nbrs M.μ κ) :=
  q.callback_unique comm

/-- The original's step from a lifted configuration, mapped down, is the quotient's step. -/
theorem h_step_lift (t : ℕ) (y : X → B) :
    q.h ∘ step nbrs (M.μ t) (q.s ∘ y) = step nbrs (q.model.μ t) y := by
  funext c
  simp only [Function.comp_apply, step, Lumping.model, List.map_map]

/-- **When the callback does not depend on what `h` forgets, lifting the whole step is the separate
construction**: the quotient's rule, then the lifted callback. No condition on the rule is needed:
the quotient's rule is defined by lifting. -/
theorem liftCallback_wholeStep {κ : ℕ → (X → M.A) → X → M.A} (hκ : q.CallbackDescends κ) :
    q.liftCallback (wholeStep nbrs M.μ κ) = wholeStep nbrs q.model.μ (q.liftCallback κ) := by
  funext n y
  show q.h ∘ κ n (step nbrs (M.μ n) (q.s ∘ y)) =
    q.h ∘ κ n (q.s ∘ step nbrs (q.model.μ n) y)
  apply hκ
  rw [q.h_comp_s_comp, q.h_step_lift]
  rfl

/-- One step from configurations that `h` does not distinguish gives results it does not
distinguish, when the rule does not depend on what it forgets. -/
theorem step_congr (hd : q.Descends) (t : ℕ) {x y : X → M.A} (e : q.h ∘ x = q.h ∘ y) :
    q.h ∘ step nbrs (M.μ t) x = q.h ∘ step nbrs (M.μ t) y := by
  funext c
  refine hd t c _ _ ?_
  simp only [List.map_map, e]

/-- **What erasing shows.** Run with what `h` forgets filled back in by the section after every
step: if neither the rule nor the callback depends on it, everything `h` keeps agrees with the
original run. So a difference disproves the map; agreement on one problem does not prove it. -/
theorem erased_run (hd : q.Descends) {κ : ℕ → (X → M.A) → X → M.A}
    (hκ : q.CallbackDescends κ) (x : X → M.A) (n : ℕ) :
    q.h ∘ runWith nbrs M.μ (fun n y ⇒ q.s ∘ (q.h ∘ κ n y)) (q.s ∘ (q.h ∘ x)) n =
      q.h ∘ runWith nbrs M.μ κ x n := by
  induction n with
  | zero ⇒ exact q.h_comp_s_comp (q.h ∘ x)
  | succ n ih ⇒
    simp only [runWith]
    rw [q.h_comp_s_comp]
    exact hκ n _ _ (q.step_congr hd n ih)

/-- **What holds of the rule and the callback separately holds of the whole step.** -/
theorem wholeStep_descends (hd : q.Descends) {κ : ℕ → (X → M.A) → X → M.A}
    (hκ : q.CallbackDescends κ) : q.CallbackDescends (wholeStep nbrs M.μ κ) :=
```

```
  fun n _ _ e ⇒ hκ n _ _ (q.step_congr hd n e)

end Lumping

/-- A model on one cell with a kept layer and a scratch layer: its rule copies the kept layer to
the scratch layer. -/
def scratchModel : CAModel PUnit.{1} where
  A := Bool × Bool
  μ _ _ l := match l with
    | [a] ⇒ (a.1, a.1)
    | _ ⇒ (false, false)

/-- Forgetting the scratch layer. -/
def scratchLumping : Lumping scratchModel Bool where
  h := Prod.fst
  s b := (b, false)
  h_s _ := rfl

/-- **The whole step strictly extends the separate construction.** A rule writes a layer that it
does not read, and a callback reads it in the same step. The rule lets `h` forget that layer and the
whole step does too, but the callback does not, so by `Lumping.callbackDescends_iff_exists` no
callback of the smaller state commutes with it. -/
theorem wholeStep_strictly_more :
    ∃ κ : ℕ → (PUnit.{1} → Bool × Bool) → PUnit.{1} → Bool × Bool,
      scratchLumping.Descends ∧ ¬ scratchLumping.CallbackDescends κ ∧
        scratchLumping.CallbackDescends (wholeStep (fun c ⇒ [c]) scratchModel.μ κ) := by
  refine ⟨fun _ x c ⇒ ((x c).2, (x c).2), ?_, ?_, ?_⟩
  · intro t c l l' e
    cases l with
    | nil ⇒
      cases l' with
      | nil ⇒ rfl
      | cons _ _ ⇒ simp at e
    | cons a l ⇒
      cases l' with
      | nil ⇒ simp at e
      | cons a' l' ⇒
        simp only [List.map_cons, List.cons.injEq] at e
        cases l with
        | nil ⇒
          cases l' with
          | nil ⇒ exact e.1
          | cons _ _ ⇒ simp at e
        | cons _ _ ⇒
          cases l' with
          | nil ⇒ simp at e
          | cons _ _ ⇒ rfl
  · intro hκ
    have := congrFun
      (hκ 0 (fun _ ⇒ (true, true)) (fun _ ⇒ (true, false)) rfl) PUnit.unit
    simp [scratchLumping] at this
  · intro n x x' e
    funext c
    exact congrFun e c
```

Block rules

A block rule does not fit `μ t c : List A → A`: a Margolus step cuts the cells into blocks, and each block's cells are written together. Here the cells are cut by an equivalence `X ≃ K × ι` (block `k`, position `i` within it), which may change from step to step, as the Margolus offsets do. The same construction applies one level up, with a block of states in place of a list.

```
/-- A block model: cell states and, at each time and for each block, a map of block states. -/
structure BlockModel (K ι : Type u) where
  /-- the cell states -/
```

```
  A : Type u
  /-- the new states of block `k` at time `t` from its states -/
  f : ℕ → K → (ι → A) → ι → A

variable {K ι : Type u}

/-- One step of a block rule: each cell takes its place in the new states of its block. -/
def blockStep {A : Type u} (e : X ≃ K × ι) (f : K → (ι → A) → ι → A) (x : X → A) : X → A :=
  fun c ⇒ f (e c).1 (fun j ⇒ x (e.symm ((e c).1, j))) (e c).2

/-- The run of a block rule, with the partition of step `t` given by `e t`. -/
def blockRun {A : Type u} (e : ℕ → X ≃ K × ι) (f : ℕ → K → (ι → A) → ι → A) (x : X → A) :
    ℕ → X → A
  | 0 ⇒ x
  | n + 1 ⇒ blockStep (e n) (f n) (blockRun e f x n)

/-- A map of block models: a map of cell states commuting with the block rules. -/
@[ext]
structure BlockMap (M N : BlockModel K ι) where
  /-- the map of cell states, applied cell by cell -/
  h : M.A → N.A
  /-- the new states of a block, mapped, are the new states from the mapped block -/
  comm : ∀ t k b, h ∘ M.f t k b = N.f t k (h ∘ b)

/-- **Maps of block models send runs to runs.** -/
theorem BlockMap.blockRun_map {M N : BlockModel K ι} (F : BlockMap M N)
    (e : ℕ → X ≃ K × ι) (x : X → M.A) (n : ℕ) :
    blockRun e N.f (F.h ∘ x) n = F.h ∘ blockRun e M.f x n := by
  induction n with
  | zero ⇒ rfl
  | succ n ih ⇒
    funext c
    simp only [blockRun, ih, blockStep, Function.comp_apply]
    exact (congrFun (F.comm n (e n c).1 _) (e n c).2).symm

/-- A way to forget something about cell states of a block model, and to fill it back in. -/
structure BlockLumping (M : BlockModel K ι) (B : Type u) where
  /-- the map of cell states -/
  h : M.A → B
  /-- the lift of a cell state -/
  s : B → M.A
  /-- lifting and mapping down changes nothing -/
  h_s : ∀ b, h (s b) = b

namespace BlockLumping

variable {M : BlockModel K ι} {B : Type u} (q : BlockLumping M B)

theorem h_comp_s_comp (y : ι → B) : q.h ∘ (q.s ∘ y) = y := by
  funext i
  exact q.h_s (y i)

/-- The quotient block model: lift the block, apply `M`'s rule, map the result down. -/
def model : BlockModel K ι where
  A := B
  f t k b := q.h ∘ M.f t k (q.s ∘ b)

/-- The new states of a block, mapped by `h`, do not depend on what `h` forgets. -/
def Descends : Prop :=
  ∀ t k (b b' : ι → M.A), q.h ∘ b = q.h ∘ b' → q.h ∘ M.f t k b = q.h ∘ M.f t k b'

/-- When the dynamics do not depend on what is forgotten, `h` is a map onto the quotient. -/
def map (hd : q.Descends) : BlockMap M q.model where
  h := q.h
  comm t k b := hd t k b (q.s ∘ (q.h ∘ b)) (q.h_comp_s_comp (q.h ∘ b)).symm
```

```
/-- **Any block rule that `h` commutes with is the quotient's**, whichever section was used. -/
theorem rule_unique {g : ℕ → K → (ι → B) → ι → B}
    (comm : ∀ t k b, q.h ∘ M.f t k b = g t k (q.h ∘ b)) : g = q.model.f := by
  funext t k b
  show g t k b = q.h ∘ M.f t k (q.s ∘ b)
  rw [comm, q.h_comp_s_comp]

/-- **`h` commutes with some block rule exactly when the dynamics do not depend on what it
forgets.** -/
theorem descends_iff_exists_rule :
    q.Descends ↔ ∃ g : ℕ → K → (ι → B) → ι → B, ∀ t k b, q.h ∘ M.f t k b = g t k (q.h ∘ b) := by
  constructor
  · intro hd
    exact ⟨q.model.f, (q.map hd).comm⟩
  · rintro ⟨g, comm⟩ t k b b' e
    rw [comm, comm, e]

/-- **The quotient runs every run of the original, mapped by `h`.** -/
theorem blockRun_quotient (hd : q.Descends) (e : ℕ → X ≃ K × ι) (x : X → M.A) (n : ℕ) :
    blockRun e q.model.f (q.h ∘ x) n = q.h ∘ blockRun e M.f x n :=
  (q.map hd).blockRun_map e x n

end BlockLumping

end CategoricalCellularAutomataProofs
```

Maps between models that read neighbours

```
import CategoricalCellularAutomataProofs.Models
import CategoricalCellularAutomataProofs.Quotients
import CategoricalCellularAutomataProofs.CurtisHedlundLyndon
import Mathlib.Data.ZMod.Basic
import Mathlib.Tactic.LinearCombination
```

A `ModelMap` sends the state of a cell to a state. A sliding map reads a finite window around the cell, so it can see what lies between cells, such as a boundary between two regions. Here it is a map of configurations `Φ` whose value at a cell depends only on the configuration on a finite window of that cell, and it is a map of models when it commutes with the steps of the two models. On a lattice these are the sliding block codes of symbolic dynamics, the package's `SlidingBlockMap`.

A sliding map sends every run to a run (`SlidingMap.run_map`). The windows of a composite are found by reading each window through the other, so sliding maps form a category (`SlidingModels`), and cell-by-cell maps are a subcategory (`ModelMap.slidingFunctor`) that misses some of them (`kinkMap126_not_cellwise`).

A rule is a family `μ t c` in the time and the cell, so counter-based random numbers are covered. A rule of the codomain may depend on the numbers of cells other than its own, as it must when the window holds several cells: in `wallsMap`, the boundaries of a voter model move on the coins of the two cells beside them, whichever coins are drawn.

Over a group, a surjective cellular automaton `Φ` has a cellular automaton `F'` with `F' ∘ Φ = Φ ∘ F` exactly when `F` respects the fibres of `Φ`, and then `F'` is unique (`factor_exists_iff`, `factor_unique`, `factor_isCA`); over a finite group that follows from the Curtis–Hedlund–Lyndon theorem.

A cell-by-cell map down can have a section that reads neighbours (`SlidingLumping`, the package's `quotient(model, h, section, section_window)`). The quotient's step lifts, steps and maps down, and it reads a finite window (`SlidingLumping.stepQ_reads`). Unlike a section of one cell, this can forget a layer that feeds back, when a cell can work it out from its neighbours. The map then holds on the consistent states, where what is forgotten agrees with the section, for as long as a run stays among them (`SlidingLumping.run_consistent`), though not on all states (`pairLumping_not_descends`). Where the dynamics do not depend on what is forgotten it holds everywhere, and this is the only step that does (`SlidingLumping.stepQ_comm`, `SlidingLumping.stepQ_unique`).

```
namespace CategoricalCellularAutomataProofs
```

```
open CategoryTheory

universe u

variable {X : Type u}

/-- A map of models that reads a window: a map of configurations, each of whose values depends only
on the configuration on a finite window of the cell, commuting with the steps of the two models. -/
@[ext]
structure SlidingMap (nbrs : X → List X) (M N : CAModel X) where
  /-- the map of configurations -/
  Φ : (X → M.A) → X → N.A
  /-- each value reads a finite window -/
  reads : ∃ w : X → List X, ∀ c x y, (∀ d ∈ w c, x d = y d) → Φ x c = Φ y c
  /-- one step followed by the map is the map followed by one step -/
  comm : ∀ t x, Φ (step nbrs (M.μ t) x) = step nbrs (N.μ t) (Φ x)

variable {nbrs : X → List X}

/-- **Sliding maps send runs to runs.** The run of the codomain from the mapped initial state is,
at every step, the run of the domain mapped. -/
theorem SlidingMap.run_map {M N : CAModel X} (f : SlidingMap nbrs M N) (x : X → M.A) (n : ℕ) :
    run nbrs N.μ (f.Φ x) n = f.Φ (run nbrs M.μ x n) := by
  induction n with
  | zero ⇒ rfl
  | succ n ih ⇒ simp only [run, ih, f.comm]

/-- The identity reads the cell alone. -/
def SlidingMap.id (M : CAModel X) : SlidingMap nbrs M M where
  Φ x := x
  reads := ⟨fun c ⇒ [c], fun c x y e ⇒ e c (by simp)⟩
  comm _ _ := rfl

/-- A sliding map followed by another reads each window of the second through the first. -/
def SlidingMap.comp {M N P : CAModel X} (f : SlidingMap nbrs M N) (g : SlidingMap nbrs N P) :
    SlidingMap nbrs M P where
  Φ x := g.Φ (f.Φ x)
  reads := by
    obtain ⟨wf, hf⟩ := f.reads
    obtain ⟨wg, hg⟩ := g.reads
    refine ⟨fun c ⇒ (wg c).flatMap wf, fun c x y e ⇒ hg c _ _ fun d hd ⇒ hf d x y fun b hb ⇒ ?_⟩
    exact e b (List.mem_flatMap.mpr ⟨d, hd, hb⟩)
  comm t x := by rw [f.comm, g.comm]

/-- The models on a space with its neighbours, for the category of sliding maps. -/
structure SlidingModels (nbrs : X → List X) where
  /-- the model -/
  model : CAModel X

/-- **Models and the sliding maps between them form a category.** -/
instance instCategorySlidingModels : Category.{u} (SlidingModels nbrs) where
  Hom M N := SlidingMap nbrs M.model N.model
  id M := SlidingMap.id M.model
  comp f g := SlidingMap.comp f g
  id_comp _ := rfl
  comp_id _ := rfl
  assoc _ _ _ := rfl

/-- A map of cell states is a sliding map, with a window of the cell alone. -/
def ModelMap.toSliding {M N : CAModel X} (f : ModelMap M N) : SlidingMap nbrs M N where
  Φ x := f.h ∘ x
  reads := ⟨fun c ⇒ [c], fun c x y e ⇒ by simp [e c (by simp)]⟩
  comm t x := (f.step_map t x).symm

/-- **Cell-by-cell maps are a subcategory of sliding maps.** -/
def ModelMap.slidingFunctor : CAModel X ⥤ SlidingModels nbrs where
```

```
  obj M := ⟨M⟩
  map f := ModelMap.toSliding f
  map_id _ := rfl
  map_comp _ _ := rfl

/-- Over a nonempty space, different maps of cell states are different sliding maps. -/
theorem ModelMap.toSliding_injective [Nonempty X] {M N : CAModel X} {f g : ModelMap M N}
    (e : (ModelMap.toSliding f : SlidingMap nbrs M N) = ModelMap.toSliding g) : f = g := by
  obtain ⟨c⟩ := ‹Nonempty X›
  ext a
  have := congrArg (fun F : SlidingMap nbrs M N ⇒ F.Φ (fun _ ⇒ a) c) e
  exact this
```

Examples on a ring

```
section Ring

variable (n : ℕ)

/-- A ring of cells, each reading the cell before it, itself and the cell after it. -/
def ringNbrs : ZMod n → List (ZMod n) := fun c ⇒ [c - 1, c, c + 1]

/-- An elementary cellular automaton on the ring, whose rule may depend on the time and cell. -/
def ringModel (r : ℕ → ZMod n → Bool → Bool → Bool → Bool) : CAModel (ZMod n) where
  A := Bool
  μ t c l := match l with
    | [a, b, d] ⇒ r t c a b d
    | _ ⇒ false

variable {n}

/-- Whether a boundary lies between a cell and the next. -/
def kink (x : ZMod n → Bool) (c : ZMod n) : Bool := xor (x c) (x (c + 1))

theorem kink_reads : ∃ w : ZMod n → List (ZMod n), ∀ c x y, (∀ d ∈ w c, x d = y d) →
    kink x c = kink y c :=
  ⟨fun c ⇒ [c, c + 1], fun c x y e ⇒ by simp [kink, e c (by simp), e (c + 1) (by simp)]⟩

/-- Elementary rule 126: a cell is on unless it and its neighbours agree. -/
def rule126 (_ : ℕ) (_ : ZMod n) (a b d : Bool) : Bool := !(a == b && b == d)

/-- Elementary rule 18: a cell that is off turns on when exactly one neighbour is on. -/
def rule18 (_ : ℕ) (_ : ZMod n) (a b d : Bool) : Bool := !b && xor a d

/-- **The boundaries of rule 126 are rule 18.** -/
def kinkMap126 : SlidingMap (ringNbrs n) (ringModel n rule126) (ringModel n rule18) where
  Φ := kink
  reads := kink_reads
  comm t x := by
    funext c
    simp only [step, ringNbrs, ringModel, List.map, kink, rule126, rule18, sub_add_cancel,
      add_sub_cancel_right]
    cases x (c - 1) <;> cases x c <;> cases x (c + 1) <;> cases x (c + 1 + 1) <;> rfl

/-- **A sliding map need not be cell by cell.** Two cells in the same state can have different
boundaries beside them. -/
theorem kinkMap126_not_cellwise [Fact (1 < n)] :
    ¬ ∃ h : Bool → Bool, ∀ (x : ZMod n → Bool) c, (kinkMap126 (n := n)).Φ x c = h (x c) := by
  rintro ⟨h, hh⟩
  have zero := hh (fun _ ⇒ false) 0
  have one := hh (fun c ⇒ decide (c = 1)) 0
  have ne : (0 : ZMod n) ≠ 1 := zero_ne_one
  simp [kinkMap126, kink, ne] at zero one
  rw [← zero] at one
```

```
  exact Bool.noConfusion one

/-- A voter model with any field of coins: a cell copies the cell before it on `true` and the cell
after it on `false`. A random draw at each cell and step is such a field. -/
def voterRule (coin : ℕ → ZMod n → Bool) (t : ℕ) (c : ZMod n) (a _ d : Bool) : Bool :=
  if coin t c then a else d

/-- The boundaries of the voter model, moved by the coins of the cells on either side. -/
def wallRule (coin : ℕ → ZMod n → Bool) (t : ℕ) (c : ZMod n) (a b d : Bool) : Bool :=
  match coin t c, coin t (c + 1) with
  | true, true ⇒ a
  | false, false ⇒ d
  | true, false ⇒ xor (xor a b) d
  | false, true ⇒ b

/-- **The boundaries between two species that compete by copying move on the coins of the two cells
beside them**, on every path, whichever coins are drawn. By `walls_need_other_coins`, no rule that
reads only its own cell's coin could do this. -/
def wallsMap (coin : ℕ → ZMod n → Bool) :
    SlidingMap (ringNbrs n) (ringModel n (voterRule coin)) (ringModel n (wallRule coin)) where
  Φ := kink
  reads := kink_reads
  comm t x := by
    funext c
    simp only [step, ringNbrs, ringModel, List.map, kink, voterRule, wallRule, sub_add_cancel,
      add_sub_cancel_right]
    cases coin t c <;> cases coin t (c + 1) <;> cases x (c - 1) <;> cases x c <;>
      cases x (c + 1) <;> cases x (c + 1 + 1) <;> rfl

/-- **A map that reads a window needs the random numbers of the window.** No rule of the boundaries
that reads the three boundaries around a cell and only that cell's coin gives their next state: on
a ring of at least four cells, the next boundary also depends on the coin of the next cell. -/
theorem walls_need_other_coins [hn : Fact (3 < n)] :
    ¬ ∃ r : ZMod n → Bool → Bool → Bool → Bool → Bool,
      ∀ (coin : ℕ → ZMod n → Bool) (x : ZMod n → Bool) (c : ZMod n),
        kink (step (ringNbrs n) ((ringModel n (voterRule coin)).μ 0) x) c =
          r c (coin 0 c) (kink x (c - 1)) (kink x c) (kink x (c + 1)) := by
  rintro ⟨r, hr⟩
  have nonzero : ∀ k : ℕ, 0 < k → k ≤ 3 → (k : ZMod n) ≠ 0 := fun k k0 k3 e ⇒ by
    have := (ZMod.natCast_eq_zero_iff k n).mp e
    have := Nat.le_of_dvd k0 this
    have := hn.out
    omega
  have one : (1 : ZMod n) ≠ 0 := by simpa using nonzero 1 (by omega) (by omega)
  have two : (2 : ZMod n) ≠ 0 := by simpa using nonzero 2 (by omega) (by omega)
  have three : (3 : ZMod n) ≠ 0 := by simpa using nonzero 3 (by omega) (by omega)
  have m1 : (-1 : ZMod n) ≠ 1 := fun e ⇒ two (by linear_combination -e)
  have m2 : (-1 : ZMod n) ≠ 2 := fun e ⇒ three (by linear_combination -e)
  have n12 : (1 : ZMod n) ≠ 2 := fun e ⇒ one (by linear_combination -e)
  let x : ZMod n → Bool := fun c ⇒ decide (c = 1) || decide (c = 2)
  have both := hr (fun _ d ⇒ decide (d = 0) || decide (d = 1)) x 0
  have only := hr (fun _ d ⇒ decide (d = 0)) x 0
  have hx : ∀ c, x c = (decide (c = 1) || decide (c = 2)) := fun _ ⇒ rfl
  simp [step, ringNbrs, ringModel, voterRule, kink, hx, one, one.symm, two.symm, m1, m2, n12,
    n12.symm, one_add_one_eq_two, zero_sub] at both only
  rw [both] at only
  simp at only

end Ring
```

Factors

```
section Factor
```

```
variable {S T : Type*}

/-- A map through a surjection exists exactly when the map respects its fibres. -/
theorem factor_exists_iff {Φ : S → T} (hs : Function.Surjective Φ) (F : S → S) :
    (∃ F' : T → T, ∀ x, F' (Φ x) = Φ (F x)) ↔ ∀ x y, Φ x = Φ y → Φ (F x) = Φ (F y) := by
  constructor
  · rintro ⟨F', h⟩ x y e
    rw [← h, ← h, e]
  · intro h
    choose s hs using hs
    refine ⟨fun y ⇒ Φ (F (s y)), fun x ⇒ h _ _ (hs _)⟩

/-- A map through a surjection is unique. -/
theorem factor_unique {Φ : S → T} (hs : Function.Surjective Φ) {F : S → S} {F₁ F₂ : T → T}
    (h₁ : ∀ x, F₁ (Φ x) = Φ (F x)) (h₂ : ∀ x, F₂ (Φ x) = Φ (F x)) : F₁ = F₂ := by
  funext y
  obtain ⟨x, rfl⟩ := hs y
  rw [h₁, h₂]

variable {G : Type u} [Group G] {A B : Type u}

/-- **The factor of a cellular automaton by a surjective one is a cellular automaton**, over a finite
group: a map on the codomain that commutes with both commutes with the shifts. -/
theorem factor_isCA [Finite G] {Φ : Config G A → Config G B} (hΦ : IsCA G Φ)
    (hs : Function.Surjective Φ) {F : Config G A → Config G A} (hF : IsCA G F)
    {F' : Config G B → Config G B} (h : ∀ x, F' (Φ x) = Φ (F x)) : IsCA G F' := by
  rw [isCA_iff_shift_comm]
  intro g y
  obtain ⟨x, rfl⟩ := hs y
  rw [← hΦ.shift_comm, h, hF.shift_comm, hΦ.shift_comm, h]

end Factor
```

Quotients by a section that reads neighbours

```
section SlidingQuotient

/-- A map of cell states with a section that reads a finite window of the smaller state. -/
structure SlidingLumping (M : CAModel X) (B : Type u) where
  /-- the map of cell states -/
  h : M.A → B
  /-- the lift of a state, which may read neighbours -/
  S : (X → B) → X → M.A
  /-- each lifted cell reads a finite window -/
  reads : ∃ w : X → List X, ∀ c y y', (∀ d ∈ w c, y d = y' d) → S y c = S y' c
  /-- the lift forgets nothing that `h` keeps -/
  h_S : ∀ y c, h (S y c) = y c

namespace SlidingLumping

variable {M : CAModel X} {B : Type u} (q : SlidingLumping M B) (nbrs : X → List X)

/-- The quotient's step: lift the state, take a step of the model, and map down. -/
def stepQ (t : ℕ) (y : X → B) : X → B := q.h ∘ step nbrs (M.μ t) (q.S y)

/-- **The quotient's step is local**: it reads the windows of the cells the model's rule reads. -/
theorem stepQ_reads (t : ℕ) : ∃ w : X → List X, ∀ c y y', (∀ d ∈ w c, y d = y' d) →
    q.stepQ nbrs t y c = q.stepQ nbrs t y' c := by
  obtain ⟨w, hw⟩ := q.reads
  refine ⟨fun c ⇒ (nbrs c).flatMap w, fun c y y' e ⇒ ?_⟩
  have : (nbrs c).map (q.S y) = (nbrs c).map (q.S y') :=
    List.map_congr_left fun d hd ⇒ hw d y y' fun b hb ⇒ e b (List.mem_flatMap.mpr ⟨d, hd, hb⟩)
  simp only [stepQ, step, Function.comp_apply, this]
```

```
/-- A consistent state: what `h` forgets is what the section would rebuild. -/
def Consistent (x : X → M.A) : Prop := q.S (q.h ∘ x) = x

/-- From a consistent state, the quotient's step is the model's step, mapped down. -/
theorem stepQ_of_consistent {x : X → M.A} (hx : q.Consistent x) (t : ℕ) :
    q.stepQ nbrs t (q.h ∘ x) = q.h ∘ step nbrs (M.μ t) x := by
  simp only [stepQ]
  rw [hx]

/-- **The quotient runs a run of the model that stays among the consistent states, mapped down.**
Nothing is needed of the dynamics on other states. -/
theorem run_consistent (x : X → M.A) (n : ℕ) (hc : ∀ k < n, q.Consistent (run nbrs M.μ x k)) :
    iterateStep (q.stepQ nbrs) (q.h ∘ x) n = q.h ∘ run nbrs M.μ x n := by
  induction n with
  | zero ⇒ rfl
  | succ n ih ⇒
    simp only [iterateStep, run]
    rw [ih fun k hk ⇒ hc k (Nat.lt_succ_of_lt hk)]
    exact q.stepQ_of_consistent nbrs (hc n (Nat.lt_succ_self n)) n

/-- A run stays among the consistent states when each step keeps them so. -/
theorem run_of_invariant {x : X → M.A} (h0 : q.Consistent x)
    (inv : ∀ t x, q.Consistent x → q.Consistent (step nbrs (M.μ t) x)) (n : ℕ) :
    iterateStep (q.stepQ nbrs) (q.h ∘ x) n = q.h ∘ run nbrs M.μ x n := by
  refine q.run_consistent nbrs x n fun k _ ⇒ ?_
  induction k with
  | zero ⇒ exact h0
  | succ k ih ⇒ exact inv k _ (ih (by omega))

/-- The dynamics do not depend on what `h` forgets. -/
def Descends : Prop := ∀ t (x x' : X → M.A), q.h ∘ x = q.h ∘ x' →
  q.h ∘ step nbrs (M.μ t) x = q.h ∘ step nbrs (M.μ t) x'

theorem h_comp_S (y : X → B) : q.h ∘ q.S y = y := funext (q.h_S y)

/-- **Where the dynamics do not depend on what is forgotten, the map holds on every state.** -/
theorem stepQ_comm (hd : q.Descends nbrs) (t : ℕ) (x : X → M.A) :
    q.stepQ nbrs t (q.h ∘ x) = q.h ∘ step nbrs (M.μ t) x :=
  hd t _ _ (q.h_comp_S _)

/-- **Any step that `h` commutes with is the quotient's.** -/
theorem stepQ_unique {G : ℕ → (X → B) → X → B}
    (comm : ∀ t x, G t (q.h ∘ x) = q.h ∘ step nbrs (M.μ t) x) : G = q.stepQ nbrs := by
  funext t y
  have := comm t (q.S y)
  rwa [q.h_comp_S] at this

end SlidingLumping

variable {n : ℕ}

/-- A state written as pairs of neighbouring cells, under the shift: each cell takes the second of
its pair, and the second of the next cell's. -/
def pairModel (n : ℕ) : CAModel (ZMod n) where
  A := Bool × Bool
  μ _ _ l := match l with
    | [_, m, r] ⇒ (m.2, r.2)
    | _ ⇒ (false, false)

/-- Forgetting the second of each pair, which the next cell can rebuild. -/
def pairLumping : SlidingLumping (pairModel n) Bool where
  h := Prod.fst
  S y c := (y c, y (c + 1))
  reads := ⟨fun c ⇒ [c, c + 1], fun c y y' e ⇒ by simp [e c (by simp), e (c + 1) (by simp)]⟩
  h_S _ _ := rfl
```

```
/-- A pair is consistent when its second is the first of the next cell's. -/
theorem pairLumping_consistent_iff {x : ZMod n → (pairModel n).A} :
    pairLumping.Consistent x ↔ ∀ c, (x c).2 = (x (c + 1)).1 := by
  constructor
  · intro h c
    exact (congrArg Prod.snd (congrFun h c)).symm
  · intro h
    funext c
    exact Prod.ext rfl (h c).symm

/-- **The quotient runs each run of consistent pairs, mapped down.** -/
theorem pairLumping_run {x : ZMod n → (pairModel n).A} (h0 : pairLumping.Consistent x) (k : ℕ) :
    iterateStep (pairLumping.stepQ (ringNbrs n)) (pairLumping.h ∘ x) k =
      pairLumping.h ∘ run (ringNbrs n) (pairModel n).μ x k := by
  refine pairLumping.run_of_invariant (ringNbrs n) h0 (fun t x hx ⇒ ?_) k
  rw [pairLumping_consistent_iff] at hx ⊢
  intro c
  simp only [step, ringNbrs, pairModel, List.map, add_sub_cancel_right]

/-- **The map holds on the consistent states and not on the others**, so a section that reads
neighbours reaches further than the condition on every state: the first of a next pair depends on
the second of this one, which `h` forgets. -/
theorem pairLumping_not_descends : ¬ pairLumping.Descends (ringNbrs n) := by
  intro hd
  have e := hd 0 (fun _ ⇒ (false, false)) (fun _ ⇒ (false, true)) rfl
  have := congrFun e 0
  simp [step, ringNbrs, pairModel, pairLumping] at this

end SlidingQuotient

end CategoricalCellularAutomataProofs
```